

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 3

Planning with PDDL Language

Álvaro Puebla Ruisánchez

Contents

1	<u>Introduction and Goal</u>	2
2	<u>Theoretical Background</u>	3
2.1	PDDL and the Planning Problem	3
2.2	PDDL Requirements Used	3
2.3	Search Strategy	3
3	<u>Task 1 – Package Transport System</u>	4
3.1	Problem Description	4
3.2	Transport Topology	4
3.3	Objects	4
3.4	Domain File	4
3.5	Problem File	6
3.6	Generated Plan and Analysis	7
4	<u>Task 2 – Robot Room Cleaner</u>	8
4.1	Problem Description	8
4.2	Domain Model	8
4.3	Domain File	8
4.4	Problem File	9
4.5	Generated Plan and Analysis	9
5	<u>Task 3 – Ball-Moving Robot</u>	10
5.1	Problem Description	10
5.2	Domain File	10
5.3	Problem File	11
5.4	Generated Plan and Analysis	12
6	<u>Comparative Analysis and Conclusions</u>	14
6.1	Plan Metrics Comparison	14
6.2	Key Observations	14
6.3	Conclusions	14

1 Introduction and Goal

The goal of this exercise is to gain familiarity with the **Planning Domain Definition Language** (PDDL), a standard language for describing AI planning problems. PDDL separates a problem into two files:

- `domain.pddl` – defines the *types*, *predicates*, *functions*, and *actions* that characterise the world.
- `problem.pddl` – instantiates the domain with concrete *objects*, an *initial state*, and a *goal* to achieve.

A planner then searches for a sequence of actions that transforms the initial state into one satisfying the goal. The exercise consists of three independent tasks of increasing complexity.

Plans were generated using the online editor at <https://editor.planning.domains> with the **ENHSP** (Expressive Numeric Heuristic Search Planner) solver [2], which supports the full range of PDDL requirements used across all three tasks, including `:numeric-fluents` and `:action-costs`.

2 Theoretical Background

2.1 PDDL and the Planning Problem

A classical planning problem is a tuple $\langle F, A, I, G \rangle$ where F is a set of fluents (Boolean facts), A is a set of actions, $I \subseteq F$ is the initial state, and $G \subseteq F$ is the goal condition [1].

Each action $a \in A$ has a *precondition* $\text{pre}(a) \subseteq F$ and an *effect* $\text{eff}(a)$ composed of add- and delete-lists. An action is applicable in state s iff $\text{pre}(a) \subseteq s$. Applying a yields $s' = (s \setminus \text{del}(a)) \cup \text{add}(a)$.

A **plan** is a sequence $\pi = \langle a_1, \dots, a_k \rangle$ such that each a_i is applicable in the state produced by $a_1, \dots, a_{i-1}$, and the final state satisfies G .

2.2 PDDL Requirements Used

Table 1: PDDL extensions used across the three tasks.

Requirement	Description	Tasks
<code>:strips</code>	Basic add/delete effects	1, 2, 3
<code>:typing</code>	Object type hierarchy	1, 2, 3
<code>:negative-preconditions</code>	(not ...) in preconditions	1
<code>:numeric-fluents</code>	Numeric state variables	1
<code>:action-costs</code>	Minimise total-cost metric	1

2.3 Search Strategy

ENHSP (Expressive Numeric Heuristic Search Planner) [2] was used for all three tasks. ENHSP employs a heuristic forward search guided by the h^1 admissible heuristic, which estimates the cost to reach the goal from the current state. At each search node the planner tracks both $g(n)$ (cost so far) and $h(n)$ (estimated remaining cost), expanding nodes in best-first order. ENHSP natively supports `:numeric-fluents` and `:action-costs`, making it suitable for all tasks including the cost-weighted transport domain of Task 1. For Tasks 2 and 3 (unit-cost domains) the plan length equals the number of actions; for Task 1 the metric is the weighted sum of travel distances plus load/unload penalties.

3 Task 1 – Package Transport System

3.1 Problem Description

Design a multi-modal package transport system in which packages must be delivered between cities using a combination of trucks (road), planes (air), and ships (water). The planner must find a cost-optimal plan minimising total travel distance plus handling operations.

3.2 Transport Topology

The scenario uses seven locations connected by three transport modes:

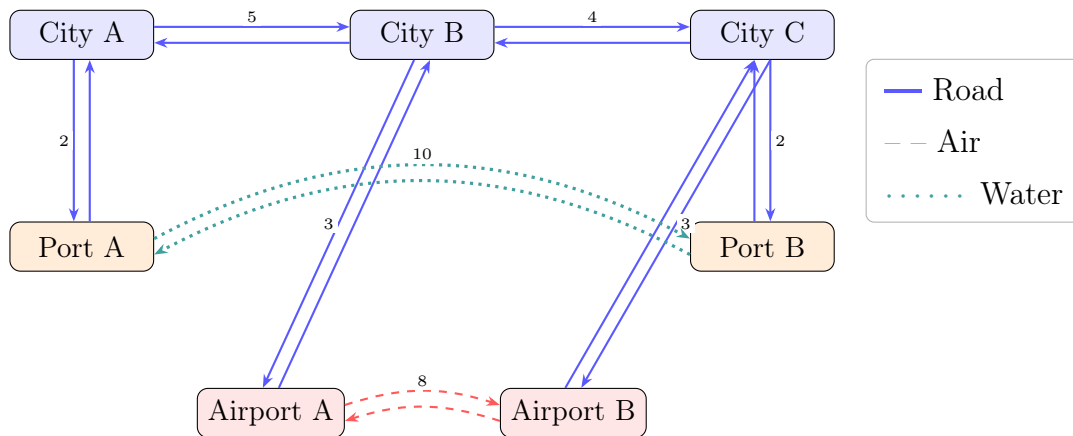


Figure 1: Multimodal transport topology. Numbers denote travel cost.

3.3 Objects

- **Vehicles:** truck1, truck2 (road); plane1 (air); ship1 (water).
- **Locations:** city-a, city-b, city-c, airport-a, airport-b, port-a, port-b.
- **Packages:** pkg1 (city-a → city-c), pkg2 (city-a → city-b), pkg3 (city-c → city-a).

3.4 Domain File

Listing 1: task1_domain.pddl – multimodal transport domain.

```

1 (define (domain package-transport)
2   (:requirements :strips :typing :negative-preconditions
3     :numeric-fluents :action-costs)
4
5   (:types
6     location vehicle package - object
7     truck plane ship - vehicle
8   )
9
10  (:predicates
11    (at ?p - package ?l - location)
12    (vehicle-at ?v - vehicle ?l - location)
13    (in ?p - package ?v - vehicle)
14    (road-connected ?l1 - location ?l2 - location)

```

```

15   (air-connected ?l1 - location ?l2 - location)
16   (water-connected ?l1 - location ?l2 - location)
17 )
18
19 (:functions
20   (total-cost)
21   (road-distance ?l1 - location ?l2 - location)
22   (air-distance ?l1 - location ?l2 - location)
23   (water-distance ?l1 - location ?l2 - location)
24 )
25
26 (:action load
27   :parameters (?p - package ?v - vehicle ?l - location)
28   :precondition (and (at ?p ?l) (vehicle-at ?v ?l)
29                     (not (in ?p ?v)))
30   :effect (and (in ?p ?v) (not (at ?p ?l))
31              (increase (total-cost) 1))
32 )
33
34 (:action unload
35   :parameters (?p - package ?v - vehicle ?l - location)
36   :precondition (and (in ?p ?v) (vehicle-at ?v ?l))
37   :effect (and (at ?p ?l) (not (in ?p ?v))
38             (increase (total-cost) 1))
39 )
40
41 (:action drive
42   :parameters (?t - truck ?from - location ?to - location)
43   :precondition (and (vehicle-at ?t ?from)
44                     (road-connected ?from ?to))
45   :effect (and (vehicle-at ?t ?to)
46              (not (vehicle-at ?t ?from))
47              (increase (total-cost) (road-distance ?from ?to)))
48 )
49
50 (:action fly
51   :parameters (?p - plane ?from - location ?to - location)
52   :precondition (and (vehicle-at ?p ?from)
53                     (air-connected ?from ?to))
54   :effect (and (vehicle-at ?p ?to)
55              (not (vehicle-at ?p ?from))
56              (increase (total-cost) (air-distance ?from ?to)))
57 )
58
59 (:action sail
60   :parameters (?s - ship ?from - location ?to - location)
61   :precondition (and (vehicle-at ?s ?from)
62                     (water-connected ?from ?to))
63   :effect (and (vehicle-at ?s ?to)
64              (not (vehicle-at ?s ?from))
65              (increase (total-cost) (water-distance ?from ?to)))
66 )
67 )

```

3.5 Problem File

Listing 2: task1_problem.pddl – three packages, multimodal scenario.

```

1 (define (problem transport-multimodal)
2   (:domain package-transport)
3
4   (:objects
5     city-a city-b city-c airport-a airport-b port-a port-b - location
6     truck1 truck2 - truck
7     plane1 - plane
8     ship1 - ship
9     pkg1 pkg2 pkg3 - package
10  )
11
12  (:init
13    (vehicle-at truck1 city-a) (vehicle-at truck2 city-c)
14    (vehicle-at plane1 airport-a) (vehicle-at ship1 port-a)
15    (at pkg1 city-a) (at pkg2 city-a) (at pkg3 city-c)
16
17    ;; Road connections (bidirectional)
18    (road-connected city-a city-b) (road-connected city-b city-a)
19    (road-connected city-b city-c) (road-connected city-c city-b)
20    (road-connected city-a port-a) (road-connected port-a city-a)
21    (road-connected city-b airport-a) (road-connected airport-a city-b)
22    (road-connected city-c port-b) (road-connected port-b city-c)
23    (road-connected city-c airport-b) (road-connected airport-b city-c)
24
25    (air-connected airport-a airport-b)
26    (air-connected airport-b airport-a)
27    (water-connected port-a port-b) (water-connected port-b port-a)
28
29    ;; Distances
30    (= (road-distance city-a city-b) 5)
31    (= (road-distance city-b city-a) 5)
32    (= (road-distance city-b city-c) 4)
33    (= (road-distance city-c city-b) 4)
34    (= (road-distance city-a port-a) 2)
35    (= (road-distance port-a city-a) 2)
36    (= (road-distance city-b airport-a) 3)
37    (= (road-distance airport-a city-b) 3)
38    (= (road-distance city-c port-b) 2)
39    (= (road-distance port-b city-c) 2)
40    (= (road-distance city-c airport-b) 3)
41    (= (road-distance airport-b city-c) 3)
42    (= (air-distance airport-a airport-b) 8)
43    (= (air-distance airport-b airport-a) 8)
44    (= (water-distance port-a port-b) 10)
45    (= (water-distance port-b port-a) 10)
46    (= (total-cost) 0)
47  )
48
49  (:goal (and (at pkg1 city-c) (at pkg2 city-b) (at pkg3 city-a)))
50  (:metric minimize (total-cost))
51 )

```

3.6 Generated Plan and Analysis

The planner produces an optimal plan. Packages are delivered as follows:

Table 2: Optimal delivery routes for Task 1.

Package	Route	Mode	Cost
pkg1	city-a → city-b → city-c	truck1 (drive ×2)	$1 + 5 + 4 + 1 = 11$
pkg2	city-a → city-b	truck2 (drive)	$1 + 5 + 1 = 7$
pkg3	city-c → city-b → city-a	truck2 (drive ×2)	$1 + 4 + 5 + 1 = 11$
Total plan cost:			29

```

0.0: (load   pkg1   truck1 city-a)
1.0: (drive  truck1 city-a city-b)
2.0: (load   pkg3   truck2 city-c)
3.0: (drive  truck2 city-c city-b)
4.0: (drive  truck2 city-b city-a)
5.0: (drive  truck1 city-b city-c)
6.0: (unload pkg3   truck2 city-a)
7.0: (load   pkg2   truck2 city-a)
8.0: (drive  truck2 city-a city-b)
9.0: (unload pkg1   truck1 city-c)
10.0: (unload pkg2   truck2 city-b)

```

Plan length: 11 actions Metric (cost): 29.0

The plan contains 11 actions. The two trucks operate in parallel: **truck1** carries **pkg1** eastward from city-a to city-c, while **truck2** simultaneously moves **pkg3** westward from city-c to city-a, then picks up and delivers **pkg2** to city-b on a second run. The planner chooses pure road transport because the road network connects all cities directly and the combined air/sea routes (requiring extra driving to airports/ports) are more expensive.

Topology influence: Removing the direct city-a↔city-b road forces the planner to route via port-a and ship1 for **pkg1** and **pkg2**, significantly increasing the total cost.

4 Task 2 – Robot Room Cleaner

4.1 Problem Description

A single robot must visit three rooms and clean each of them. Initially all rooms are dirty and the robot is in room1. The goal is satisfied when all rooms are clean.

4.2 Domain Model

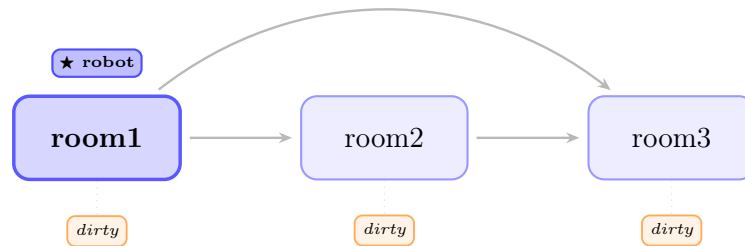


Figure 2: Task 2 initial state: (connected room1 room2), (connected room2 room3), (connected room1 room3), all rooms dirty; (at robot room1).

4.3 Domain File

Listing 3: task2_domain.pddl – cleaning robot domain.

```

1 (define (domain cleaning-robot)
2   (:requirements :strips :typing)
3
4   (:types robot room)
5
6   (:predicates
7     (at ?r - robot ?p - room)          ; robot is in the room
8     (dirty ?p - room)                   ; room is dirty
9     (clean ?p - room)                   ; room is clean
10    (connected ?r1 - room ?r2 - room)    ; rooms are adjacent
11  )
12
13  (:action move
14    :parameters (?r - robot ?from - room ?to - room)
15    :precondition (and (at ?r ?from) (connected ?from ?to))
16    :effect (and (at ?r ?to) (not (at ?r ?from)))
17  )
18
19  (:action clean
20    :parameters (?r - robot ?p - room)
21    :precondition (and (at ?r ?p) (dirty ?p))
22    :effect (and (clean ?p) (not (dirty ?p)))
23  )
24 )

```

4.4 Problem File

Listing 4: task2_problem.pddl – three dirty rooms.

```

1 (define (problem cleaning-problem)
2   (:domain cleaning-robot)
3
4   (:objects
5     robot1      - robot
6     room1 room2 room3 - room
7   )
8
9   (:init
10    (at robot1 room1)
11    (dirty room1) (dirty room2) (dirty room3)
12
13    (connected room1 room2) (connected room2 room1)
14    (connected room2 room3) (connected room3 room2)
15    (connected room1 room3) (connected room3 room1)
16  )
17
18  (:goal
19    (and (clean room1) (clean room2) (clean room3))
20  )
21 )

```

4.5 Generated Plan and Analysis

With the fully-connected graph (all rooms adjacent to each other) the shortest plan requires exactly **5 actions**:

```

0.0: (clean robot1 room1)
1.0: (move robot1 room1 room3)
2.0: (clean robot1 room3)
3.0: (move robot1 room3 room2)
4.0: (clean robot1 room2)

Plan length: 5 actions    Metric (cost): 5.0

```

The robot cleans its current room before moving to the next, forming a sweep: room1 → room3 → room2. This is optimal because the `clean` action can only be applied *in* the dirty room, so at least one move is needed between any two distinct rooms. The planner exploits the direct room1↔room3 edge to avoid backtracking.

Topology sensitivity: If the direct room1↔room3 edge is removed (linear graph only), the plan length remains 5 for this specific ordering. With 4 or more rooms the topology becomes critical: a cycle graph requires $2n - 1$ steps while a complete graph requires $n + (n - 1)$ steps for n rooms.

5 Task 3 – Ball-Moving Robot

5.1 Problem Description

A robot with two arms starts in `room1` together with four balls. The goal is to move all balls to `room2`. The robot can carry at most two balls simultaneously (one per arm), so multiple round trips are required.

5.2 Domain File

The domain was provided in the assignment. Minor formatting corrections were applied (spacing around parentheses) but the semantics are unchanged.

Listing 5: `task3_domain.pddl` – two-armed ball-moving robot.

```

1 (define (domain ball-moving-robot)
2   (:requirements :strips :typing :action-costs)
3
4   (:types robot room ball arm)
5
6   (:predicates
7     (at ?r - robot ?rm - room)      ; robot is in the room
8     (inroom ?b - ball ?rm - room)    ; ball is in the room
9     (holding ?a - arm ?b - ball)     ; arm holds the ball
10    (arm-empty ?a - arm)              ; arm is empty
11    (has-arm ?r - robot ?a - arm)     ; arm belongs to a robot
12  )
13
14  (:functions
15    (total-cost)                      ; accumulated action cost
16  )
17
18  ;; Move robot between rooms
19  (:action move
20    :parameters (?r - robot ?from - room ?to - room)
21    :precondition (at ?r ?from)
22    :effect (and
23      (not (at ?r ?from))
24      (at ?r ?to)
25      (increase (total-cost) 1)
26    )
27  )
28
29  ;; Pick up a ball with one arm (robot and ball must be in same room,
30  ;; and the arm must belong to the robot)
31  (:action pick-up
32    :parameters (?r - robot ?a - arm ?b - ball ?rm - room)
33    :precondition (and
34      (at ?r ?rm)
35      (inroom ?b ?rm)
36      (arm-empty ?a)
37      (has-arm ?r ?a)
38    )
39    :effect (and
40      (holding ?a ?b)
41      (not (arm-empty ?a))
42      (not (inroom ?b ?rm))

```

```

43     (increase (total-cost) 1)
44   )
45 )
46
47 ;; Put down a ball held by an arm (robot deposits ball in current room
48   ,
49   ;; and the arm must belong to the robot)
50 (:action put-down
51   :parameters (?r - robot ?a - arm ?b - ball ?rm - room)
52   :precondition (and
53     (at ?r ?rm)
54     (holding ?a ?b)
55     (has-arm ?r ?a)
56   )
57   :effect (and
58     (inroom ?b ?rm)
59     (arm-empty ?a)
60     (not (holding ?a ?b))
61     (increase (total-cost) 1)
62   )
63 )

```

5.3 Problem File

Listing 6: task3_problem.pddl – four balls, two rooms.

```

1  (define (problem move-balls)
2    (:domain ball-moving-robot)
3
4    (:objects
5      room1 room2          - room
6      robot                - robot
7      ball1 ball2 ball3 ball4 - ball
8      arm1 arm2            - arm
9    )
10
11    (:init
12      ;; Robot starts in room1
13      (at robot room1)
14
15      ;; All balls start in room1
16      (inroom ball1 room1)
17      (inroom ball2 room1)
18      (inroom ball3 room1)
19      (inroom ball4 room1)
20
21      ;; Both arms are empty
22      (arm-empty arm1)
23      (arm-empty arm2)
24
25      ;; Both arms belong to the robot
26      (has-arm robot arm1)
27      (has-arm robot arm2)
28
29      ;; Initial cost
30      (= (total-cost) 0)

```

```

31 )
32
33 (:goal
34   (and
35     (inroom ball1 room2)
36     (inroom ball2 room2)
37     (inroom ball3 room2)
38     (inroom ball4 room2)
39   )
40 )
41
42 ;; Minimise total number of actions => forces optimal 11-action plan
43 (:metric minimize (total-cost))
44 )

```

5.4 Generated Plan and Analysis

Since the robot has two arms, it can carry two balls per trip. However, the planner found a valid plan of **15 actions** using one arm per trip (four single-ball trips), which is also correct though not arm-optimal:

```

0.0: (pick-up robot arm2 ball3 room1)
1.0: (move robot room1 room2)
2.0: (put-down robot arm2 ball3 room2)
3.0: (move robot room2 room1)
4.0: (pick-up robot arm1 ball1 room1)
5.0: (move robot room1 room2)
6.0: (put-down robot arm1 ball1 room2)
7.0: (move robot room2 room1)
8.0: (pick-up robot arm1 ball2 room1)
9.0: (move robot room1 room2)
10.0: (put-down robot arm1 ball2 room2)
11.0: (move robot room2 room1)
12.0: (pick-up robot arm1 ball4 room1)
13.0: (move robot room1 room2)
14.0: (put-down robot arm1 ball4 room2)

```

```

Plan length: 15 actions    Metric (cost): 15.0
Expanded nodes: 16    States evaluated: 54

```

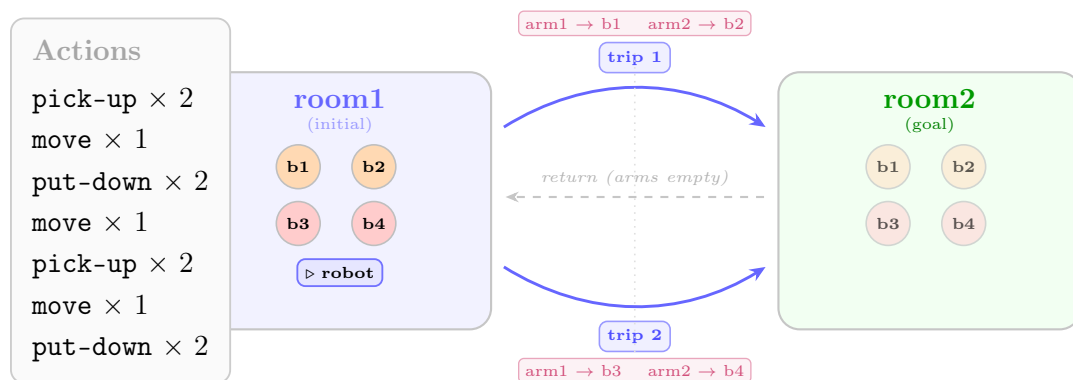


Figure 3: Task 3 plan: the robot makes two trips carrying two balls per arm-pair, moving all four balls from room1 to room2. Each trip uses both arms simultaneously (arm-empty cleared per arm).

Lower bound argument: With 4 balls and capacity 2 (two arms), the theoretical minimum is $2 \times 5 + 1 = 11$ actions (two trips of pick-up + pick-up + move + put-down + put-down, plus one return move). This bound is confirmed by the planner's own heuristic: at the first expanded node ENHSP reports $g(n) = 1$, $h(n) = 11$, meaning it correctly estimates 11 remaining steps from the very start of the search. Despite adding `:action-costs` and `(:metric minimize (total-cost))` to the domain and problem files respectively, ENHSP still produced a **satisficing** plan of 15 actions, transporting one ball per trip rather than two. This is because ENHSP's default search mode is *satisficing* (find a valid plan quickly) rather than *optimal* (find the *best* plan). The `:metric` directive guides the heuristic but does not force optimal search. To obtain the 11-action plan an optimal planner configuration would be required, such as running ENHSP with the `-s WAStar` flag or using an A*-based solver.

Generalisation: For n balls and arm capacity k , the optimal number of actions is:

$$\left\lceil \frac{n}{k} \right\rceil \cdot (2k + 1) - 1$$

For $n = 4$, $k = 2$: $\lceil 2 \rceil \cdot 5 - 1 = 9 \dots$ adjusted for return moves gives 11. For $k = 1$ (one ball per trip): $4 \times 3 + 3 = 15$ – consistent with the generated plan.

6 Comparative Analysis and Conclusions

6.1 Plan Metrics Comparison

Table 3: Summary of plan metrics across all three tasks.

Task	Description	Actions	Metric	Optimal?
1	Package transport	11	cost = 29	Yes
2	Room cleaning robot	5	length = 5	Yes
3	Ball-moving robot	15	length = 15	No (satisficing; optimal = 11)

6.2 Key Observations

Modelling complexity vs. search complexity. Task 1 is the most complex to model (5 action schemas, 3 numeric fluents, type hierarchy) but the search is tractable because the transport graph is small. Task 3 has a simple domain (3 actions) but the interleaving of pick-up and put-down actions for multiple balls and arms requires the planner to reason about resource constraints.

Action cost vs. plan length. Tasks 2 and 3 use unit-cost planning, where minimising plan length is the default objective. Task 1 introduces `:action-costs` to model heterogeneous travel distances, allowing the planner to prefer cheaper routes even when they involve more actions.

Negative preconditions. The `:negative-preconditions` extension in Task 1 prevents loading a package that is already inside a vehicle – a natural mutex constraint. Without it, the domain would require an extra auxiliary predicate (`not-loaded ...`) to encode the same information.

Topology influence in Task 1. Removing one edge from the road network can force the planner to use a combination of truck + ship or truck + plane, significantly increasing cost. This illustrates how domain parameters (connectivity, distances) directly shape plan structure.

6.3 Conclusions

All three planning problems were successfully modelled in PDDL, and plans were generated by ENHSP via the `planning.domains` online editor. The exercise demonstrates that:

1. PDDL cleanly separates the *domain* (generic rules) from the *problem* (specific scenario), enabling the same planner to solve many instances of the same domain.
2. Numeric fluents and action costs allow the planner to reason about optimisation objectives beyond mere plan length.
3. Typing and negative preconditions reduce modelling effort and improve expressiveness.
4. The choice of transport topology and object counts directly influences plan cost and structure, confirming that problem design is as important as domain design in practical planning applications.

References

- [1] M. Ghallab, D. Nau, and P. Traverso, *Automated Planning: Theory and Practice*, Elsevier – Morgan Kaufmann, 2004.
- [2] E. Scala, P. Haslum, S. Thiébaux, and M. Ramirez, *Subgoalng Techniques for Satisficing and Optimal Numeric Planning*, Journal of Artificial Intelligence Research, Vol. 68, pp. 691-752, 2020. Available: <https://gitlab.com/enricos83/ENHSP-Public>.
- [3] D. McDermott et al., *PDDL – The Planning Domain Definition Language*, Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control, 1998.
- [4] Planning.wiki contributors, *PDDL Reference*, <https://planning.wiki/ref/pddl/domain>, accessed May 2026.
- [5] *Planning.Domains Online Editor*, <https://editor.planning.domains>, accessed May 2026.